

第031讲 一维梯度下降独立实验指南

1 实验问题与独立答案

本实验用同一条已知答案的二次曲线，检验“走对方向”为什么仍会过冲、循环或发散。必须先独立得到答案，再用程序观察真实浮点更新；不使用优化器自己的终点作为正确性证据。

五点数据是 $x=(0,1,2,3,4)$ 、 $y=(1,2,2,4,6)$ 。截距随斜率更新为 $a(b)=3-2b$ ，使用目标 $SSE/(2n)$ ，得到 $f(b)=4/25+(b-6/5)^2$ 。曲率 $q=2$ ，唯一最小点 $6/5$ ，最小值 $4/25$ 。程序对这个消去截距后的标量问题迭代，不能把结果解释成同时更新截距和斜率的二维GD。

先用纸笔写出 $\eta=1/4$ 的前两轮。 $b_0=0$ 时 $a_0=3$ ，预测全为3，残差 $(2,1,1,-1,-3)$ ，目标 $8/5$ ；导数贡献 $r_i(x_i-2)/5$ 相加为 $-12/5$ 。更新到 $b_1=3/5$ 、 $a_1=9/5$ ，重新预测得到损失 $13/25$ 、导数 $-6/5$ ；再更新到 $b_2=9/10$ 、 $a_2=6/5$ ，损失 $1/4$ 。每次更新后必须重做前向，不能把旧导数连续使用两次。

2 文件 环境与执行方式

lecture.pdf是完整正文，answers.pdf是18题完整解答，本文件是可以单独执行的实验路线。experiment.py只依赖Python标准库；Notebook为作图使用NumPy和Matplotlib。data中三份输入全部是本课程原创合成数据或实验配置，没有外部下载、账号、GPU或付费API。

推荐在本讲目录创建独立环境：

```
conda env create -f environment.yml
conda activate ml-course-031
python experiment.py
jupyter lab
```

已有Python3.12时也可安装requirements.txt。脚本的默认输入和默认输出以脚本自身目录为准；手动传入相对路径则以当前工作目录为准。

```
python -m pip install -r requirements.txt
python experiment.py --output outputs/my_result.json
python -O experiment.py --output outputs/optimized_result.json
```

本版本实际使用Python3.12.14、NumPy2.3.5、Matplotlib3.10.8、nbformat5.11.1和ipykernel7.4.0执行。Notebook在新的Python进程中使用真实InProcessKernel顺序运行并保存输出。Anaconda安装、浏览器Jupyter界面和独立进程socket传输没有实际测试；上面的环境文件是复现说明，不是对所有本地系统的保证。

3 先检查输入 再相信任何输出

regression.csv包含id、x、y；id唯一，x/y为有限实数。learning_rates.csv用整数分子和分母保存精确学习率。model_spec.json包含初值、固定轨迹长度、停止预算、梯度阈值、参数误差阈值和两个缩放常数。必须把所有行与所有字段都验证完，包括最后一条暂时不画的学习率，然后才运行第一条轨迹。

```
import experiment as e
xs, ys = e.read_data()
rates = e.read_rates()
spec = e.read_spec()
report = e.compile_report(xs, ys, rates, spec)
print(report['profile']['exact'])
```

应得到 $q=2$ 、 $center=6/5$ 、 $constant=4/25$ 、 $intercept=3/5$ 。profile使用实际输入float存储值的Fraction参照；默认输入恰好都是可精确表示的整数。随后solver把这些系数转换为binary64。请分别打印`1.2.as_integer_ratio()`和`Fraction(6,5)`，确认存储中心与数学 $6/5$ 不同。不要把小数打印相同当成精确相同。

Notebook的固定叙述和图名依赖完整默认数据。它会先校验三份输入的完整SHA256，任何改动都在首次数值输出与画图前拒绝。研究自定义配置请使用脚本，或明确同步改写Notebook的推导、图和结论；不要直接删掉守卫后保留旧答案。

4 逐行重做两轮前向与梯度累积

先不用trace函数，自己实现下面三行关系： $\hat{y}_i=3+b(x_i-2)$ ， $r_i=\hat{y}_i-y_i$ ， $g=\sum r_i(x_i-2)/5$ 。打印每行的 $\hat{y}$ 、 r 、 r^2 、局部导数 (x_i-2) 和梯度贡献 $r_i(x_i-2)/5$ 。

```
b = 0.0
for t in range(3):
    pred = [3 + b * (x - 2) for x in xs]
    residual = [p - y for p, y in zip(pred, ys)]
    contributions = [r * (x - 2) / 5 for r, x in zip(residual, xs)]
    loss = sum(r * r for r in residual) / 10
    gradient = sum(contributions)
    print(t, b, pred, residual, contributions, loss, gradient)
    b -= 0.25 * gradient
```

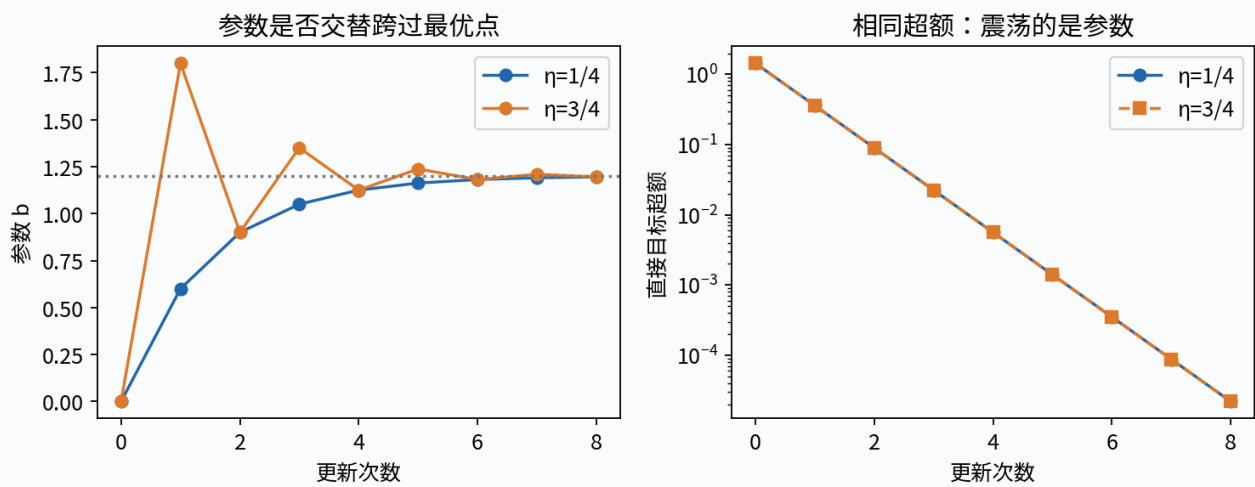
这一小段仅针对固定手算例子，程序通用接口的输入检查不在这里重写。核对第0轮梯度-2.4，第1轮-1.2，第2轮-0.6；目标依次1.6、0.52、0.25。若梯度大五倍，检查是否漏除样本数；若得到-8.4，检查是否误用固定截距下的导数 x_i 。

5 固定20轮 看清参数与损失的差别

默认七种学习率是0、1/10、1/4、1/2、3/4、1、5/4。每条fixed轨迹有初值 $t=0$ 和最多20次真实更新；只有明确的growth_guard可以提前结束，并保留实际完成数，不能截短后仍画成完整20轮。

```
for item in report['rates']:
    path = item['fixed']
    print(item['label'], item['rho_exact'], path['status'],
          path['updates_completed'], path['final']['w'])
```

图上至少同时检查参数 w 、误差绝对值、直接超额和完整目标。 $\eta=1/4$ 从同一侧靠近， $\eta=3/4$ 在两侧交替；两者在精确算术里每轮超额完全相同，都是上一轮的1/4，所以“参数震荡”不等于“损失忽高忽低”。 $\eta=1$ 时0与12/5循环，完整目标恒为8/5，不能被“目标变化0”骗过。



实验图1 两条误差倍率相反但绝对值相同的轨迹，对应相同的精确目标超额。

对数坐标不接受0。Notebook把精确0的超额显示在标明的 $1e-18$ 下限，只为可视化，原始JSON仍保存0；这不能被解释成计算残差恰好等于 $1e-18$ 。数值路径与精确路径在后期可能有舍入差异，应结合记录解释。

6 三套参照必须分开

rational_theory使用原始整数/分数定义的精确模型；stored_model_exact_arithmetic使用Fraction.from_float读取的实际系数，但之后以精确算术递推；fixed.history包含每轮真实float乘法和减法。第一套与第二套的差异来自系数表示，第二套与第三套的差异来自迭代舍入，二者不能混成一句“理论不准”。

自行用Fraction重复前四轮，不要抄程序输出到expected数组。对每轮验证 $e_t = \rho^t e_0$ ，以及目标变化等于 $-\eta(1-\eta q/2)g_t^2$ 。至少核对 $\eta=0$ 、1/2、1这三个边界和已经在最优点的初值。

对照记录first_threshold：以 $\eta=1/4$ 达到参数误差 10^{-6} 需21轮，达到梯度 10^{-10} 需35轮。必须再检验20和34轮不满足，否则“第一轮满足”缺少证据。exact_thresholds以配置中实际float阈值的精确二进制值为准；正文的十进制分数阈值在本例给出相同整数，不能一般假定总是相同。

7 缩放目标与改变参数单位

观察`report['scaling']`。将目标乘10时，同样 $\eta=1/4$ 使 ρ 从 $1/2$ 变成 -4 ；补偿为 $\eta=1/40$ ，原单位参数路径恢复。这里改变的是目标的数值量级，最小点没有变。

再令 $v=100w$ ，新目标曲率为 $2/10000$ 。匹配原路径的学习率为2500，而不是25。画图前把 v 除100再与 w 比较；若直接把两个数值放在同一参数坐标轴上，会把单位变化误认为算法变化。同一个数字 $\eta=.25$ 在 v 坐标中对应原坐标的极小更新，所以会显得几乎不动。

```
base = report['scaling']['base']['history']
scaled = report['scaling']['coordinate_matched']['history']
print(max(abs(a['w'] - b['w'] / 100) for a, b in zip(base, scaled)))
```

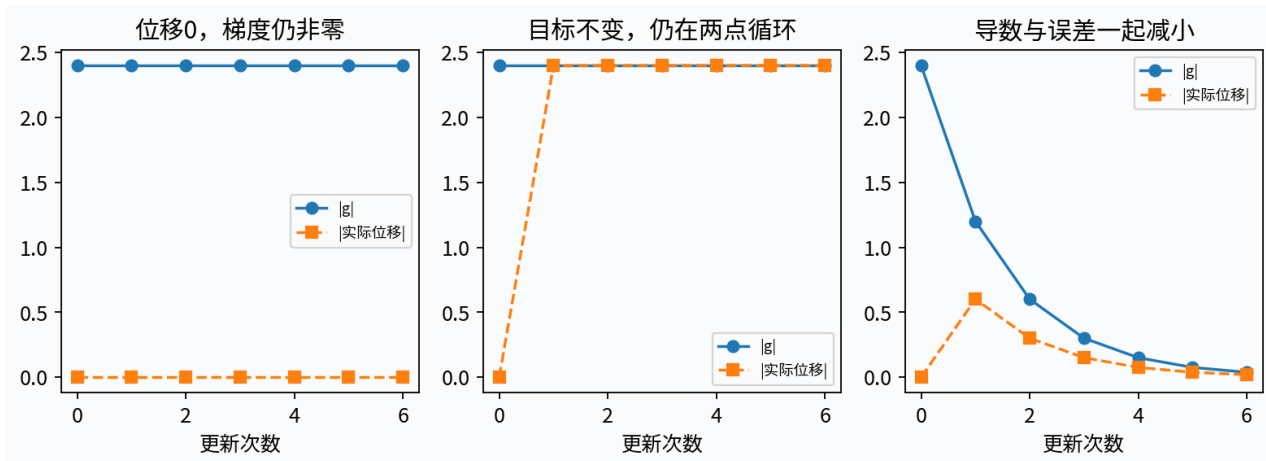
这次应在浮点舍入精度附近匹配，不要求所有机器上的逐位相等。原始参数路径恢复不代表目标数值相同：目标乘10后完整目标和超额都乘10。

8 停止状态与浮点陷阱

stopping模式首先计算当前状态，再检查是否等于存储中心或满足梯度阈值，然后检查预算，最后决定更新。预算0也保留初始状态；初值已经在存储最优点时是`exact_stored_optimum`，非最优时是`max_steps`。这个状态名指实际存储二次模型，不声称`float1.2`等于数学 $6/5$ 。

stagnation表示下一参数会与当前float完全相同；可能来自 $\eta=0$ ，也可能来自步长小于浮点间隔。two_cycle要求下一值恰好回到上一个端点；本例会把这个端点加入历史再停止。growth_guard在候选超过既定幅度前保护性停止，最后记录仍是最后一个合法状态。fixed_budget_completed只是完整执行了指定轮数，不是收敛证书。运行中的非有限或下溢计算会明确报数值范围错误，不保存部分成功报告。

在`cancellation_example`中， w 取存储中心上方的相邻float。直接计算 $q - e^2/2$ 仍为正，先加 c 再减 c 却可能得到0。再把学习率设成 $1e-10$ 、梯度阈值设成 $1e-30$ ，可看到梯度尚非零却浮点停滞。请解释它与“初始就在最优点”的区别。



实验图2 微小位移、相同损失与可靠梯度标准检查的是不同对象。

9 必做失败实验

把原始数据或配置复制到临时目录后修改，不覆盖教学输入。依次尝试：最后一行y写1e-400、最后一条rate分母0、布尔max_steps、重复JSON键、字符串counterexample_rate、重复行id，以及二维列表传入profile。原始非零数字下溢为0必须拒绝；真正的0e-400可以作为数值0接受，但具体字段仍需满足正值要求。

为每个例子先放一个写有SENTINEL的旧报告。失败后旧字节应完全不变；再把输出指向不存在的新目录，失败后该目录仍不应创建。最后用python -O重做，证明输入门槛不依赖会被移除的assert。

直接API拒绝bool、复数、文本、Fraction/Decimal外部实数与扩展精度标量；明确的exact_trace函数才接受Fraction。普通Python/NumPy有限实数在受支持范围内可以使用。输入绝对值一般 $\leq 1e8$ ，非零值 $\geq 1e-100$ ；计算的非零次正规数与正量下溢为0被拒绝。数据最多1000行，学习率最多30条，普通预算最多10000，固定展示预算最多1000；这些是教学实现的资源与数值边界。

10 超出二次函数的反例与提交

单独手算 w^4 从 $w=1$ 、 $\eta=1$ 更新：导数4，下一点-3，函数值从1变81。这个反例没有违背二次模型定理，因为全实数范围内曲率不是固定有限常数。再写出 $-w^2$ 和 w^3 在0的导数，说明只检测零梯度为什么不够。

提交重新运行的Notebook、JSON报告、自行实现的逐行两轮核对、参数与超额图、两个缩放对照、停止状态解释和坏输入保护记录。所有结论限于明确的模型、浮点实现和有限试验，不把CPU教学曲线解读成真实任务泛化或大模型训练性能。